

KPTC Scheduler

はじめての独立Linuxサーバー構築手順書

専門家でなくても、内部スケジューラーと外部空き状況ページを2台のUbuntu Serverへ順番に構築できる実作業ガイド

この版の特徴

入力値を最初に記入し、各操作を「どの端末で」「何を実行し」「何が表示されれば成功か」の順で説明します。失敗した場合は、その場で確認するコマンドと戻る章を示します。

対象	固定する条件	完成後
内部サーバー	Ubuntu Server 24.04 LTS / Nginx / PHP-FPM	社内LANまたはVPNから予定を閲覧・編集
外部サーバー	Ubuntu Server 24.04 LTS / Nginx / PHP-FPM	インターネットから3室の直近3か月空き状況を閲覧
データ連携	内部から外部への署名付きHTTPSだけ	外部側に内部DBや個人情報を置かない

文書版 v3.0 更新日 2026-08-17 対象 main a4469ae

この手順書でできること

この資料は、現在GitHubのmainに保存されているKPTC Schedulerを、新しい2台のLinuxサーバーへ配置するためのものです。

完成する構成

職員のブラウザ	内部サーバー	外部サーバー	一般のブラウザ
予定を閲覧・編集	画面 + PHP API SQLite DB ログイン認証	空き状況画面 3か月JSON DBなし	3室の空き状況を閲覧
社内LAN / VPN	内部から外部へHTTPS送信 →	← 内部への接続は行わない	インターネット

重要

PDFを上から機械的に実行するのではなく、青い「正常なら」欄を毎回確認してから次へ進みます。赤い「停止」欄が出た場合は、その先へ進まないでください。

この資料で扱わない作業

- ・サーバー購入、ラック設置、電源・ネットワーク配線
- ・組織固有のDNS登録、VPN構築、正式なTLS証明書の発行承認
- ・既存サーバーの廃止判断や個人情報保護規程の承認

ここだけは管理担当者へ依頼

内部・外部のDNS名、内部アクセス可能範囲、外部サーバーの公開可否、正式なHTTPS証明書の4点は、組織のネットワーク管理担当者を確認してください。

02 / 開始判定

作業を始められる条件

次の表をすべて確認してください。1つでも「いいえ」があれば、先にサーバー管理担当者へ相談します。

確認	必要な状態	判定
サーバー2台	内部用と外部用にUbuntu Server 24.04 LTSが1台ずつある	☐ はい
管理権限	両サーバーへSSH接続でき、sudoコマンドを実行できる	☐ はい
内部接続	職員PCから内部サーバーの80/443番へ接続できる	☐ はい
外部接続	外部サーバーの80/443番をインターネットへ公開できる	☐ はい
DNS	内部用と外部用のFQDNが決まっている	☐ はい
時刻同期	両サーバーで時刻同期が有効。ずれは5分未満	☐ はい
バックアップ	内部DBのバックアップ先が決まっている	☐ はい
作業時間	初回は2〜4時間の停止・確認時間を確保している	☐ はい

作業を中止する条件

SSH接続が切れる、sudoが使えない、ディスク残量が2GB未満、既存Webサイトが同じサーバーにあり影響範囲が不明、正式なバックアップがない場合は作業を停止します。

この資料で使う3つの画面

- 作業用PCのターミナル: GitHubからファイルを取得し、2台へ送る
- 内部サーバーのSSH画面: スケジューラーを構築する
- 外部サーバーのSSH画面: 空き状況ページを構築する

見分け方

各コマンドの直前に「作業用PC」「内部サーバー」「外部サーバー」のどこで実行するかを明記しています。別の画面へ貼り付けしないでください。

03 / 記入シート

最初に実際の値を書き込む

このページを印刷するか、別の安全なメモへ転記します。秘密鍵とパスワードだけはこのPDFへ書き込まず、組織の秘密管理場所へ保存してください。

項目	記入する値	記入例
内部サーバーSSH接続先		admin@10.20.1.10
外部サーバーSSH接続先		admin@203.0.113.20
内部FQDN		scheduler.intra.example.jp
外部FQDN		availability.example.jp
内部アクセス許可範囲		10.20.0.0/16
初期管理者アカウントID		admin
紐づけるメンバーID		m1
リリース名	20260817-a4469ae	この版では変更しない
Gitコミット	a4469ae5e04722cd7b6084f8de3610dc689882c2	この版では変更しない
内部DB保存先	/var/lib/kptc-scheduler/group-watcher.sqlite	このまま使用
外部JSON保存先	/var/lib/kptc-availability/public-availability.json	このまま使用

共有秘密鍵

第10章で内部サーバー上に生成します。64文字の英数字が表示されますが、このページには書かず、パスワード管理システムなどへ保存してください。内部・外部に同じ値を1回ずつ設定します。

メンバーIDについて

新規DBの初期データでは m1 が佐藤 美咲、m6〜m8が試験室です。既存DBを移行する場合は、管理者にする人の実際のメンバーIDを既存環境で確認します。

04 / 作業の順番

8つの到達点

チェックポイントを1つずつ通過します。途中で中断する場合は、最後に完了した番号を記録してください。

到達点	完了の目印	チェック
A ファイル準備	2つのtar.gzが作業用PCにある	☐
B 内部基盤	NginxとPHP-FPMがactive	☐
C 内部画面	ログイン画面がブラウザーに表示	☐
D 初期管理者	管理者でログインし、予定表を閲覧	☐
E 内部HTTPS	鍵マーク付きHTTPSでログイン	☐
F 外部画面	空き状況ページがHTTPSで表示	☐
G JSON連携	3室・3か月の情報が外部画面へ表示	☐
H 自動再送	2つのtimerがactive、healthがok	☐

先に内部、次に外部

内部DBの初期化時点では外部送信が失敗しても予定表は動作します。外部サーバー完成後に手動送信すると、再送待ちは解消されます。

本番データを移行する場合

まず空の新規DBでA〜Hを完了し、動作を確認します。その後、第25章「既存データの移行」を実施してください。最初から本番DBを使わない方が安全です。

05 / A ファイル準備

GitHubから実行用ファイルを取得する

現在のmainには、さくらサーバーと内容一致を確認した実行用スナップショットがあります。初心者向け手順ではこれを利用し、Node.jsによるビルド作業を省略します。

STEP A-1 Gitとファイルを取得

実行する場所

作業用PCのターミナル

コピーして実行

```
cd ~/Documents
git clone https://github.com/Apfelengineer/260725_GW_WEB.git
cd 260725_GW_WEB
git checkout a4469ae5e04722cd7b6084f8de3610dc689882c2
```

正常なら

最後の行付近に detached HEAD の案内が出ても問題ありません。
続けて `git log -1 --oneline` を実行し、先頭が `a4469ae` なら成功です。

注意

すでに同名フォルダがある場合は、新しい空フォルダで実行してください。既存フォルダを削除しないでください。

STEP A-2 25ファイルの一致を確認

実行する場所

作業用PCのターミナル。macOSは上、Linuxは下のどちらか一方

コピーして実行

```
cd server-runtime-snapshot

# macOS
shasum -a 256 -c SHA256SUMS

# Linux
# sha256sum -c SHA256SUMS
```

正常なら

25行すべてが OK と表示されます。1行でも FAILED があれば中止し、GitHubから取り直します。

06 / A ファイル準備

内部用・外部用の2つへ梱包する

さくら専用の.user.iniはNginxで不要なため、梱包から除外します。

STEP A-3 2つの配布ファイルを作成

実行する場所

作業用PCのターミナル。server-runtime-snapshotフォルダ内

コピーして実行

```
tar --exclude='.user.ini' -C schedule -czf ../kptc-internal-20260817-a4469ae.tar.gz .
tar --exclude='.user.ini' -C calendar -czf ../kptc-public-20260817-a4469ae.tar.gz .
cd ..
ls -lh kptc-*-20260817-a4469ae.tar.gz
```

正常なら

内部用が約2MB、外部用が約300KBの2ファイルとして表示されます。容量は多少異なっても構いません。

STEP A-4 サーバーへ転送

実行する場所

作業用PCのターミナル。接続先を記入シートの値へ置換

コピーして実行

```
scp kptc-internal-20260817-a4469ae.tar.gz \
  admin@10.20.1.10:/tmp/

scp kptc-public-20260817-a4469ae.tar.gz \
  admin@203.0.113.20:/tmp/

scp deploy/kptc-availability-*.service deploy/kptc-availability-*.timer \
  admin@10.20.1.10:/tmp/
```

正常なら

各コマンドが100%で終了し、エラーが表示されません。初回接続時の yes/no は、接続先アドレスを再確認してから yes を入力します。

注意

admin、10.20.1.10、203.0.113.20は例です。必ず記入シートの値へ変更します。

内部サーバーへ必要なソフトを入れる

ここから第15章までは、必ず内部サーバーのSSH画面で作業します。

STEP B-1 OSと空き容量を確認

実行する場所
内部サーバーのSSH画面
コピーして実行
<pre>cat /etc/os-release df -h / timedatectl status</pre>
正常なら
Ubuntu 24.04、空き容量2GB以上、System clock synchronized: yes を確認します。
注意
条件が違う場合はこの手順を続けず、サーバー管理者へ確認してください。

STEP B-2 Nginx・ PHP・ SQLite関連をインストール

実行する場所
内部サーバーのSSH画面
コピーして実行
<pre>sudo apt update sudo apt install -y nginx php-fpm php-cli php-sqlite3 php-curl php-mbstring \ sqlite3 curl openssl php -v ls /run/php/php*-fpm.sock sudo systemctl is-active nginx</pre>
正常なら
PHP 8.3系が表示され、/run/php/php8.3-fpm.sockのようなファイルと、最後にactiveが表示されます。ソケット名を記入シートへ追記します。

08 / B 内部基盤

内部ファイルとDB保存場所を作る

Web画面とデータベースを別の場所へ置きます。WebからDBを直接ダウンロードできない構成です。

STEP B-3 ディレクトリを作成

実行する場所

内部サーバーのSSH画面

コピーして実行

```
sudo install -d -o root -g root -m 0755 /opt/kptc-scheduler/releases
sudo install -d -o root -g root -m 0755 /opt/kptc-scheduler/internal
sudo install -d -o www-data -g www-data -m 0750 /var/lib/kptc-scheduler
sudo install -d -o root -g www-data -m 0750 /etc/kptc-scheduler
sudo install -d -o root -g root -m 0700 /var/backups/kptc-scheduler
```

正常なら

コマンドが何も表示せず終了します。

STEP B-4 内部用tarを展開

実行する場所

内部サーバーのSSH画面

コピーして実行

```
sudo install -d -m 0755 /opt/kptc-scheduler/releases/20260817-a4469ae
sudo tar -xzf /tmp/kptc-internal-20260817-a4469ae.tar.gz \
  -C /opt/kptc-scheduler/releases/20260817-a4469ae
sudo ln -sfn /opt/kptc-scheduler/releases/20260817-a4469ae \
  /opt/kptc-scheduler/internal/current
test -f /opt/kptc-scheduler/internal/current/index.html
test -f /opt/kptc-scheduler/internal/current/api.php
echo $?
```

正常なら

最後に0と表示されます。0以外ならtarの転送・展開をやり直します。

到達点 B 前半

□ /opt/kptc-scheduler/internal/current に index.html と api.php があります。

09 / B 内部設定

共有秘密鍵を生成し、内部設定を書く

共有秘密鍵は内部から外部へ送るJSONの改ざん検知に使います。画面ログインのパスワードとは別物です。

STEP B-5 共有秘密鍵を1回だけ生成

実行する場所

内部サーバーのSSH画面

コピーして実行

```
openssl rand -hex 32
```

正常なら

64文字の英数字が1行表示されます。安全なパスワード管理場所へコピーします。

注意

この値をGitHub、メール、チャット、スクリーンショットへ保存しないでください。外部設定が終わるまで安全に保持します。

09 / B 内部設定 / 続き

STEP B-6 内部環境設定を作成

実行する場所

内部サーバーのSSH画面

ファイルを開く

```
sudo nano /etc/kptc-scheduler/internal-server.env
```

次の内容を貼り付けます。外部FQDNと SHARED_SECRET を実値へ変更します。初期確認まではCookieをHTTPでも使えるよう、末尾を0にします。

貼り付ける内容

```
KPTC_INTERNAL_SCHEDULER_DB=/var/lib/kptc-scheduler/group-watcher.sqlite
KPTC_PUBLIC_AVAILABILITY_MODE=https
KPTC_PUBLIC_AVAILABILITY_ENDPOINT=https://availability.example.jp/calendar/receive-availability.php
KPTC_PUBLIC_AVAILABILITY_PAGE_URL=https://availability.example.jp/calendar
KPTC_PUBLIC_AVAILABILITY_SECRET=SHARED_SECRET
KPTC_PUBLIC_AVAILABILITY_TIMEOUT=10
KPTC_PUBLIC_AVAILABILITY_STALE_SECONDS=1800
KPTC_SESSION_COOKIE_SECURE=0
KPTC_AUTH_IDLE_TIMEOUT=1800
KPTC_AUTH_ABSOLUTE_TIMEOUT=43200
```

nanoでは Control+O、Enter、Control+X の順で保存して終了します。

権限を設定

```
sudo chown root:www-data /etc/kptc-scheduler/internal-server.env
sudo chmod 0640 /etc/kptc-scheduler/internal-server.env
sudo -u www-data test -r /etc/kptc-scheduler/internal-server.env
echo $?
```

正常なら

最後に0と表示されます。

10 / B 内部設定

PHPが環境設定を読めるようにする

Nginxから実行するPHPは、Web公開領域外の設定ファイルを介して秘密値を読み込みます。

STEP B-7 内部PHP設定ファイルを作成

ファイルを開く

```
sudo nano /etc/kptc-scheduler/internal-env.php
```

そのまま貼り付ける内容

```
<?php
$values = parse_ini_file(
    '/etc/kptc-scheduler/internal-server.env',
    false,
    INI_SCANNER_RAW
);
foreach ($values as $key => $value) {
    putenv($key . '=' . $value);
}
```

Control+O、Enter、Control+Xで保存します。

権限とPHP構文を確認

```
sudo chown root:www-data /etc/kptc-scheduler/internal-env.php
sudo chmod 0640 /etc/kptc-scheduler/internal-env.php
sudo -u www-data php -l /etc/kptc-scheduler/internal-env.php
```

正常なら

No syntax errors detected と表示されます。

秘密値を表示しない確認方法

`sudo -u www-data test -r /etc/kptc-scheduler/internal-env.php; echo $?` が0なら読み取り可能です。内容を画面へ表示するcatコマンドは使いません。

11 / C 内部画面

内部Nginxを初期HTTPで設定する

最初はLAN内HTTPで画面とDB初期化を確認します。完成後の第15章でHTTPSへ切り替えます。

STEP C-1 Nginx設定ファイルを作成

ファイルを開く

```
sudo nano /etc/nginx/sites-available/kptc-scheduler
```

FQDN、CIDR、PHPソケットの3箇所を記入シートの値へ変更します。

貼り付ける内容

```
server {
    listen 80;
    server_name scheduler.intra.example.jp;
    root /opt/kptc-scheduler/internal/current;
    index index.html;

    allow 10.20.0.0/16;
    deny all;

    location / {
        try_files $uri $uri/ /index.html;
    }

    location = /api.php {
        include snippets/fastcgi-php.conf;
        fastcgi_pass unix:/run/php/php8.3-fpm.sock;
        fastcgi_param KPTC_INTERNAL_CONFIG_FILE /etc/kptc-scheduler/internal-env.php;
    }

    location ~ \.php$ { return 404; }
}
```

Control+O、Enter、Control+Xで保存します。

有効化して確認

```
sudo ln -sf /etc/nginx/sites-available/kptc-scheduler \
    /etc/nginx/sites-enabled/kptc-scheduler
sudo unlink /etc/nginx/sites-enabled/default 2>/dev/null || true
sudo nginx -t
sudo systemctl reload nginx
sudo systemctl is-active nginx
```

正常なら

syntax is ok、test is successful、active の3つを確認します。

403 Forbiddenの場合

アクセス元PCのIPがallowで指定した範囲外です。CIDRを勝手に0.0.0.0/0へ広げず、ネットワーク担当者へ正しい範囲を確認します。

12 / C 内部画面

DBを初期化し、ログイン画面を確認する

ブラウザを開く前に、APIへ1回アクセスしてSQLiteと初期メンバーを作成します。

STEP C-2 APIを初期化

実行する場所

内部サーバーのSSH画面。FQDNを実値へ変更

コピーして実行

```
curl -sS http://scheduler.intra.example.jp/api.php?action=bootstrap
ls -l /var/lib/kptc-scheduler/group-watcher.sqlite
sudo -u www-data sqlite3 \
  /var/lib/kptc-scheduler/group-watcher.sqlite \
  "PRAGMA integrity_check;
```

正常なら

JSON内に "authenticated":false と "setupRequired":true、最後に ok が表示されます。

注意

この時点では外部サーバーが未完成のため、PHPログに外部送信失敗が記録されても問題ありません。

STEP C-3 ブラウザーで表示

操作する場所

社内LANまたはVPNへ接続した職員PCのブラウザ

開くURL

http://scheduler.intra.example.jp/ (内部FQDNへ変更)

正常なら

KPTC Schedulerのログイン画面と「管理者による初期設定が必要」という案内が表示されます。白紙の場合は第28章を確認します。

到達点 C

□ ブラウザーにKPTC Schedulerのログイン画面が表示されています。

13 / D 初期管理者

管理者アカウントを安全に作る

パスワードをコマンド履歴に残さず、標準入力から登録します。

STEP D-1 管理者を作成

実行する場所

内部サーバーのSSH画面。adminとm1を記入シートの値へ変更

コピーして実行

```
read -s KPTC_NEW_ADMIN_PASSWORD
printf '%s' "$KPTC_NEW_ADMIN_PASSWORD" | \
sudo -u www-data env \
  KPTC_INTERNAL_CONFIG_FILE=/etc/kptc-scheduler/internal-env.php \
  php /opt/kptc-scheduler/internal/current/manage-auth-user-cli.php \
  create admin m1 admin --password-stdin
unset KPTC_NEW_ADMIN_PASSWORD
```

入力方法

1行目の後、画面に文字が表示されない状態で初期管理者パスワードを入力しEnterを押します。表示されないのは正常です。

正常なら

Created: admin のように表示されます。

対応するユーザーが見つからない場合

m1などのメンバーIDが誤っています。新規DBならm1を使用します。既存DB移行時は、正しいmember_idを確認してから再実行します。

STEP D-2 管理者ログイン

- ブラウザーを再読み込みします。
- ユーザープルダウンから作成した管理者を選びます。
- 登録したパスワードを入力し、ログインします。
- 週表示の予定表と「ユーザー」「予定種別」管理が表示されることを確認します。

到達点 D

□ 管理者でログインし、予定表、ユーザー管理、予定種別管理を閲覧できます。□ ゲストとしてログインし直すと閲覧のみになります。

内部サーバー単体の確認を完了する

外部サーバーへ進む前に、内部の権限とデータ保存を確認します。

試験	操作	正常結果
予定追加	管理者で明日の予定を1件追加	再読み込み後も残る
一般権限	一般ユーザーを作成してログイン	予定編集可、ユーザー管理不可
試験室権限	room権限を設定	ログイン候補に表示されず、閲覧専用
ゲスト	ログアウトしてゲストログイン	予定は見えるが追加・編集不可
再起動	sudo systemctl restart php8.3-fpm nginx	再読み込み後も予定が残る
DB権限	ls -l /var/lib/kptc-scheduler/	DB、-wal、-shmをwww-dataが読書き可能

内部ログを確認

```
sudo journalctl -u nginx -n 30 --no-pager
sudo journalctl -u php8.3-fpm -n 30 --no-pager
```

正常なら

fatal、permission denied、PHP Parse errorがありません。外部サーバー未完成による送信エラーはこの段階では許容します。

15 / E 内部HTTPS

正式なHTTPSへ切り替える

ログイン情報を守るため、本番利用ではHTTPのまま運用しません。内部証明書の発行方式は組織により異なります。

最初に依頼すること

ネットワーク管理者へ内部FQDN用の証明書一式を依頼します。fullchain形式の証明書と秘密鍵を受け取り、ファイル名を kptc-scheduler.fullchain.pem と kptc-scheduler.key にそろえて /tmp へ転送します。

証明書と秘密鍵を保護して配置

内部サーバーで実行

```
sudo install -o root -g root -m 0644 /tmp/kptc-scheduler.fullchain.pem \  
  /etc/ssl/certs/kptc-scheduler.fullchain.pem  
sudo install -o root -g root -m 0600 /tmp/kptc-scheduler.key \  
  /etc/ssl/private/kptc-scheduler.key  
sudo openssl x509 -in /etc/ssl/certs/kptc-scheduler.fullchain.pem \  
  -noout -subject -issuer -dates
```

正常なら

subjectに内部FQDNが含まれ、notBeforeとnotAfterが現在有効な期間です。違う場合は設定せず、証明書の発行元へ確認します。

15 / E 内部HTTPS / 続き

STEP E-1 Nginx設定を完成形へ置き換え

ファイルを開く

```
sudo nano /etc/nginx/sites-available/kptc-scheduler
```

FQDN、CIDR、PHPソケットを記入シートの値へ置換えし、次の全内容で上書きします。

貼り付ける完成形

```
server {
    listen 80;
    server_name scheduler.intra.example.jp;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name scheduler.intra.example.jp;
    root /opt/kptc-scheduler/internal/current;
    index index.html;
    ssl_certificate /etc/ssl/certs/kptc-scheduler.fullchain.pem;
    ssl_certificate_key /etc/ssl/private/kptc-scheduler.key;

    allow 10.20.0.0/16;
    deny all;

    location / {
        try_files $uri $uri/ /index.html;
    }
    location = /api.php {
        include snippets/fastcgi-php.conf;
        fastcgi_pass unix:/run/php/php8.3-fpm.sock;
        fastcgi_param KPTC_INTERNAL_CONFIG_FILE /etc/kptc-scheduler/internal-env.php;
    }
    location ~ \.php$ { return 404; }
}
```

CookieをHTTPS専用へ戻して反映

内部サーバーで実行

```
sudo sed -i 's/KPTC_SESSION_COOKIE_SECURE=0/KPTC_SESSION_COOKIE_SECURE=1/' \
/etc/kptc-scheduler/internal-server.env
sudo nginx -t
sudo systemctl reload nginx
sudo systemctl reload php8.3-fpm
```

到達点 E

□ <https://内部FQDN/> を開くと証明書警告がなく、鍵マークが表示され、管理者ログインできます。警告がある状態で本番運用しません。

外部サーバーへ必要なソフトを入れる

ここから第20章までは外部サーバーのSSH画面で作業します。内部DBは外部へコピーしません。

STEP F-1 OS・時刻・空き容量を確認

実行する場所
外部サーバーのSSH画面
コピーして実行
<pre>cat /etc/os-release df -h / timedatectl status</pre>
正常なら
Ubuntu 24.04、空き容量1GB以上、時刻同期yesを確認します。

STEP F-2 Nginx・PHP・Certbotをインストール

実行する場所
外部サーバーのSSH画面
コピーして実行
<pre>sudo apt update sudo apt install -y nginx php-fpm php-cli php-mbstring curl certbot \ python3-certbot-nginx php -v ls /run/php/php*-fpm.sock sudo systemctl is-active nginx</pre>
正常なら
PHP 8.3系、PHP-FPMソケット、activeを確認します。

17 / F 外部基盤

外部画面とJSON保存場所を作る

外部側はSQLiteを使いません。内部から受け取る単一JSONだけをWeb公開領域外へ保存します。

STEP F-3 ディレクトリを作成して展開

実行する場所

外部サーバーのSSH画面

コピーして実行

```
sudo install -d -o root -g root -m 0755 /opt/kptc-availability/releases
sudo install -d -o root -g root -m 0755 /opt/kptc-availability/public
sudo install -d -o www-data -g www-data -m 0700 /var/lib/kptc-availability
sudo install -d -o root -g www-data -m 0750 /etc/kptc-availability
sudo install -d -m 0755 /opt/kptc-availability/releases/20260817-a4469ae
sudo tar -xzf /tmp/kptc-public-20260817-a4469ae.tar.gz \
-C /opt/kptc-availability/releases/20260817-a4469ae
sudo ln -sfn /opt/kptc-availability/releases/20260817-a4469ae \
/opt/kptc-availability/public/current
test -f /opt/kptc-availability/public/current/index.html
test -f /opt/kptc-availability/public/current/receive-availability.php
echo $?
```

正常なら

最後に0と表示されます。

外部へ置いてはいけないもの

api.php、auth.php、manage-auth-user-cli.php、group-watcher.sqliteが外部のcurrentに存在しないことを確認します。

存在しないことを確認

```
test ! -f /opt/kptc-availability/public/current/api.php
test ! -f /opt/kptc-availability/public/current/auth.php
find /opt/kptc-availability -name '*.sqlite' -print
```

最後のfindが何も表示しなければ正常です。

18 / F 外部設定

外部サーバーへ同じ共有秘密鍵を設定する

第9章で生成した共有秘密鍵を使用します。新しい鍵を作ってはいけません。

STEP F-4 外部環境設定を作成

ファイルを開く

```
sudo nano /etc/kptc-availability/external-server.env
```

貼り付ける内容

```
KPTC_PUBLIC_AVAILABILITY_JSON=/var/lib/kptc-availability/public-availability.json
KPTC_PUBLIC_AVAILABILITY_SECRET=SHARED_SECRET
KPTC_PUBLIC_AVAILABILITY_STALE_SECONDS=1800
```

SHARED_SECRETを第9章と同じ64文字へ置換し、Control+O、Enter、Control+Xで保存します。

権限を設定

```
sudo chown root:www-data /etc/kptc-availability/external-server.env
sudo chmod 0640 /etc/kptc-availability/external-server.env
```

STEP F-5 外部PHP設定ファイルを作成

ファイルを開く

```
sudo nano /etc/kptc-availability/public-env.php
```

そのまま貼り付ける内容

```
<?php
$values = parse_ini_file(
    '/etc/kptc-availability/external-server.env',
    false,
    INI_SCANNER_RAW
);
foreach ($values as $key => $value) {
    putenv($key . '=' . $value);
}
```

保存後に確認

```
sudo chown root:www-data /etc/kptc-availability/public-env.php
sudo chmod 0640 /etc/kptc-availability/public-env.php
sudo -u www-data php -l /etc/kptc-availability/public-env.php
```

正常なら

No syntax errors detected と表示されます。

19 / F 外部画面

外部Nginxを設定する

最初にHTTPで構文と画面を確認し、その直後にCertbotでHTTPSへ切り替えます。

STEP F-6 Nginx設定ファイルを作成

ファイルを開く

```
sudo nano /etc/nginx/sites-available/kptc-availability
```

外部FQDNとPHPソケットを実値へ変更します。

貼り付ける内容

```
server {
    listen 80;
    server_name availability.example.jp;

    location = /calendar { return 301 /calendar/; }
    location = /calendar/ {
        alias /opt/kptc-availability/public/current/index.html;
    }
    location = /calendar/index.html {
        alias /opt/kptc-availability/public/current/index.html;
    }
    location = /calendar/reservations.html {
        alias /opt/kptc-availability/public/current/reservations.html;
    }
    location /calendar/assets/ {
        alias /opt/kptc-availability/public/current/assets/;
    }
    location = /calendar/technology-center-logo-white.png {
        alias /opt/kptc-availability/public/current/technology-center-logo-white.png;
    }
    location ~ ^/calendar/(receive-availability|public-availability|health-availability)\.php$ {
        include fastcgi_params;
        fastcgi_param SCRIPT_FILENAME /opt/kptc-availability/public/current/$1.php;
        fastcgi_param KPTC_PUBLIC_CONFIG_FILE /etc/kptc-availability/public-env.php;
        fastcgi_pass unix:/run/php/php8.3-fpm.sock;
        client_max_body_size 128k;
    }
    location ~ \.php$ { return 404; }
}
```

20 / F 外部HTTPS

外部画面を有効化して証明書を取得する

外部DNSがこのサーバーを指し、80番と443番へ接続できる状態で実施します。

STEP F-7 Nginx設定を有効化

実行する場所

外部サーバーのSSH画面

コピーして実行

```
sudo ln -sf /etc/nginx/sites-available/kptc-availability \
/etc/nginx/sites-enabled/kptc-availability
sudo unlink /etc/nginx/sites-enabled/default 2>/dev/null || true
sudo nginx -t
sudo systemctl reload nginx
curl -I http://availability.example.jp/calendar/
```

正常なら

Nginxのtest successfulと、HTTP/1.1 200 OKが表示されます。FQDNは実値へ変更します。

STEP F-8 Let's Encrypt証明書を設定

実行する場所

外部サーバーのSSH画面。FQDNを実値へ変更

コピーして実行

```
sudo certbot --nginx -d availability.example.jp
sudo certbot renew --dry-run
curl -I https://availability.example.jp/calendar/
```

正常なら

証明書取得成功、dry run成功、最後にHTTP/2 200またはHTTP/1.1 200が表示されます。ブラウザでも証明書警告がありません。

注意

Certbotが失敗する場合は、DNSと80番の公開状態を確認します。証明書警告を無視して本番公開しません。

到達点 F

□ <https://外部FQDN/calendar/> に空き状況ページが表示されます。この段階では「データなし」でも正常です。

21 / G JSON連携

内部から外部へ最初の空き状況を送る

ここだけは内部サーバーへ戻って作業します。外部サーバーから内部DBへ接続する操作はありません。

STEP G-1 手動送信

実行する場所

内部サーバーのSSH画面

コピーして実行

```
sudo -u www-data env \  
  KPTC_INTERNAL_CONFIG_FILE=/etc/kptc-scheduler/internal-env.php \  
  php /opt/kptc-scheduler/internal/current/publish-availability-cli.php
```

正常なら

`{"ok":true, ...}` が1行表示され、コマンドがエラーなく終了します。

注意

署名エラー401なら両側の共有秘密鍵、接続エラーなら外部FQDN・証明書・ファイアウォール、409なら時刻同期を確認します。

STEP G-2 外部側の保存状態を確認

実行する場所

外部サーバーのSSH画面。FQDNを実値へ変更

コピーして実行

```
sudo ls -l /var/lib/kptc-availability/public-availability.json  
curl -sS https://availability.example.jp/calendar/health-availability.php  
curl -sS https://availability.example.jp/calendar/public-availability.php
```

正常なら

JSONファイルがwww-data所有・0600で存在し、healthに `"ok":true`、公開読取にroomsとdaysだけが表示され、利用者名・件名・メモが含まれません。最後にブラウザで3室を切り替え、当月を含む3か月が表示されることを確認します。

22 / H 自動再送

5分ごとの再送と10分ごとの監視を有効にする

予定保存直後の送信に失敗しても、自動的に再送できるようにします。

STEP H-1 serviceとtimerを配置

実行する場所

内部サーバーのSSH画面

コピーして実行

```
sudo install -o root -g root -m 0644 /tmp/kptc-availability-publish.service \
/etc/systemd/system/
sudo install -o root -g root -m 0644 /tmp/kptc-availability-publish.timer \
/etc/systemd/system/
sudo install -o root -g root -m 0644 /tmp/kptc-availability-monitor.service \
/etc/systemd/system/
sudo install -o root -g root -m 0644 /tmp/kptc-availability-monitor.timer \
/etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now kptc-availability-publish.timer
sudo systemctl enable --now kptc-availability-monitor.timer
```

正常なら

2つのtimerについてCreated symlinkが表示されます。すでに有効な場合は表示されなくても構いません。

STEP H-2 timerと監視結果を確認

実行する場所

内部サーバーのSSH画面

コピーして実行

```
systemctl list-timers 'kptc-availability-*'
sudo systemctl start kptc-availability-publish.service
sudo systemctl status kptc-availability-publish.service --no-pager
sudo -u www-data env \
  KPTC_INTERNAL_CONFIG_FILE=/etc/kptc-scheduler/internal-env.php \
  php /opt/kptc-scheduler/internal/current/monitor-availability-cli.php
```

正常なら

timer一覧にpublishとmonitorがあり、publish serviceはstatus=0/SUCCESS、監視JSONは "ok":true です。oneshot serviceがinactiveになっていても直前結果がSUCCESSなら正常です。

到達点 H

□ publishは5分、monitorは10分間隔で登録され、手動実行と監視が成功します。

利用開始前の総合チェック

すべてにチェックが付くまで利用開始を案内しません。

分類	完成条件	結果
内部HTTPS	証明書警告なし。社内LAN/VPN以外から接続不可	☐ 合格
管理者	ログイン、予定操作、ユーザー管理が可能	☐ 合格
一般	予定操作可能、ユーザー追加・削除不可	☐ 合格
試験室	閲覧のみ。ログイン候補へ表示しない	☐ 合格
ゲスト	パスワードなし閲覧、変更操作不可	☐ 合格
内部DB	Web公開領域外。再起動後も予定が残る	☐ 合格
外部HTTPS	インターネットから空き状況ページを表示	☐ 合格
公開内容	3室・3か月・4状態のみ。個人情報なし	☐ 合格
署名送信	publish成功、health ok、30分以内	☐ 合格
自動再送	timer 2件が有効、ログに連続失敗なし	☐ 合格
バックアップ	内部DBの日次バックアップと復元試験を確認	☐ 合格
再起動	両サーバー再起動後も全機能が復帰	☐ 合格

最後のヘルスチェック

```
#
sudo systemctl is-active nginx php8.3-fpm
sudo systemctl is-enabled kptc-availability-publish.timer
sudo systemctl is-enabled kptc-availability-monitor.timer

#
sudo systemctl is-active nginx php8.3-fpm
curl -fsS https://availability.example.jp/calendar/health-availability.php
```

利用開始

すべて合格した日時、担当者、Gitコミット a4469ae、内部URL、外部URLを運用記録へ残します。

24 / 日常運用

バックアップと確認を自動化する

内部DBだけが復旧不能な業務データです。外部JSONは内部から再生成できます。

毎日: 内部DBバックアップ

内部サーバーで実行する例

```
sudo sqlite3 /var/lib/kptc-scheduler/group-watcher.sqlite \  
  ".backup '/var/backups/kptc-scheduler/group-watcher-$(date +%F).sqlite'" \  
sudo sqlite3 /var/backups/kptc-scheduler/group-watcher-$(date +%F).sqlite \  
  "PRAGMA integrity_check;"
```

正常なら

integrity_checkがokです。バックアップには認証ハッシュと操作履歴が含まれるため、暗号化された管理領域へ保管します。

毎週: 監視ログ確認

内部サーバー

```
journalctl -u kptc-availability-publish.service --since '7 days ago' --no-pager \  
journalctl -u kptc-availability-monitor.service --since '7 days ago' --no-pager
```

毎月: 復元できるか確認

- バックアップを別の試験用フォルダへコピーします。
- PRAGMA integrity_checkがokであることを確認します。
- 四半期ごとに別ホストでログイン・予定・JSON送信まで復元試験します。

外部JSON

履歴保存は不要です。壊れた場合はファイルを退避して内部publishを再実行します。

さくら環境の予定を新内部サーバーへ移す

新規構築の確認が全て完了した後だけ実施します。変更凍結中に整合したSQLiteバックアップを作ります。

順番	作業	成功確認
1	利用者へ変更停止時間を案内	予定登録が止まっている
2	現行DBと-wal/-shmの扱いを統一して.backup	integrity_checkがok
3	新内部サーバーへ安全に転送	/tmpにのみ一時配置
4	PHP-FPMを止めてDBを入れ替え	所有者www-data、権限0660
5	PHP-FPMを起動しAPIへアクセス	自動移行が完了
6	管理者・予定・履歴を確認	件数と内容が一致
7	publishを手動実行	外部3か月が更新
8	利用者確認後に凍結解除	新URLだけを案内

新内部サーバーでDBを入れ替える例

```
sudo systemctl stop php8.3-fpm
sudo cp /var/lib/kptc-scheduler/group-watcher.sqlite \
/var/backups/kptc-scheduler/before-migration.sqlite
sudo install -o www-data -g www-data -m 0660 /tmp/group-watcher.sqlite \
/var/lib/kptc-scheduler/group-watcher.sqlite
sudo systemctl start php8.3-fpm
curl -sS https://scheduler.intra.example.jp/api.php?action=bootstrap
```

必ずバックアップ

元DBと新規確認用DBの両方を残します。問題があればPHP-FPMを止め、before-migration.sqliteへ戻します。

将来のバージョン更新と戻し方

新しいファイルは別releaseフォルダへ展開し、currentリンクだけを切り替えます。

更新前

- GitHubで承認済みコミットを決める
- 内部DBをバックアップし、integrity_checkを実行する
- 新しい内部・外部tarを別releaseフォルダへ展開する
- すべてのPHPへphp -lを実行する

切り替え

内部サーバーの例

```
sudo ln -sf /opt/kptc-scheduler/releases/NEW_RELEASE \
/opt/kptc-scheduler/internal/current
sudo systemctl reload php8.3-fpm nginx
```

外部サーバーの例

```
sudo ln -sf /opt/kptc-availability/releases/NEW_RELEASE \
/opt/kptc-availability/public/current
sudo systemctl reload php8.3-fpm nginx
```

戻す場合

- currentリンクを直前のreleaseへ戻します。
- DB変更後に問題がある場合は書き込みを止め、更新前バックアップへ戻します。
- 外部JSONは互換なら保持し、必要なら内部から再送します。

削除しない

新旧releaseと更新前DBは、動作確認と規定の保存期間が終わるまで削除しません。

症状から確認場所を探す

エラーメッセージをそのまま記録し、秘密値を伏せて管理担当者へ伝えます。

症状	最初に確認	戻る章
SSH接続できない	IP、ユーザー名、VPN、22番FW	02～03
nginx -tが失敗	表示されたファイル名と行番号、括弧・セミコロン	11 / 19
403 Forbidden	内部CIDRのallow、接続元IP	11
502 Bad Gateway	PHP-FPM active、ソケット名	07 / 16
白紙ページ	assetsが200、ブラウザーConsole、index.html	28
APIが500	PHPログ、DB権限、internal-env.php構文	08～12
管理者作成失敗	DB初期化、member_id、設定ファイル	12～13
ログイン後すぐ戻る	HTTPS、Cookie Secure、時刻	15
外部データなし	public JSON、手動publish、health	21
受信401	共有秘密鍵と両サーバー時刻	09 / 18 / 21
受信409	sourceVersion、rangeStart、古いJSON	21
health 503	最終送信30分以内、当月rangeStart	21～22

内部ログ

```
sudo journalctl -u nginx -n 100 --no-pager
sudo journalctl -u php8.3-fpm -n 100 --no-pager
sudo journalctl -u kptc-availability-publish.service -n 100 --no-pager
```

外部ログ

```
sudo journalctl -u nginx -n 100 --no-pager
sudo journalctl -u php8.3-fpm -n 100 --no-pager
```

共有時の注意

ログ内の共有秘密鍵、Cookie、Authorization、利用者名、予定件名、メモ、IPアドレスは必要に応じて伏せます。

画面が何も表示されない場合

index.htmlはReact画面の入口です。JavaScriptが読み込めないとroot要素が空のままになり、白紙に見えます。

STEP 1 ファイルを直接開いていないか

- URLが file:///.../index.html なら誤りです。必ず http:// または https:// のサーバーURLを開きます。

STEP 2 静的ファイルのHTTP応答

内部FQDNへ変更して実行

```
curl -I https://scheduler.intra.example.jp/  
curl -I https://scheduler.intra.example.jp/assets/main-CaRfBn_1.css  
curl -I https://scheduler.intra.example.jp/assets/main-ujX_p4eN.js
```

正常なら

3つとも200です。404ならcurrentリンク、root、assetsフォルダを確認します。

STEP 3 API応答

内部FQDNへ変更して実行

```
curl -i https://scheduler.intra.example.jp/api.php?action=bootstrap
```

正常なら

HTTP 200とJSONが返ります。HTML、404、500ならNginxの/api.php設定、PHP-FPM、internal-env.php、DB権限を確認します。

STEP 4 ブラウザー開発者ツール

- Consoleの赤いエラーとNetworkの失敗URLを記録します。
- CORS、Failed to fetch、Unexpected tokenがあればURLとAPI応答を確認します。

index.htmlだけでは動かない

実行用ファイル一式をローカルでダブルクリックしても、PHPとDBが動かないためスケジューラーにはなりません。

利用開始前に守ること

初心者向けに操作を具体化しても、秘密値と内部データを守る原則は変わりません。

対象	必須ルール	確認
GitHub	DB、共有秘密鍵、env、JSON、秘密鍵をコミットしない	☐
内部Web	社内LAN/VPNだけ許可。HTTPS必須	☐
外部Web	内部API、auth、SQLiteを置かない	☐
共有秘密鍵	32文字以上。内部・外部だけに0640で保存	☐
管理者	空パスワード禁止。十分に長い固有パスワード	☐
バックアップ	暗号化、アクセス制限、復元試験	☐
ログ	秘密値と個人情報を外部へ送らない	☐
更新	承認済みコミット、ハッシュ、ロールバック準備	☐
証明書	自動更新と期限監視	☐
時刻	NTP同期。送受信のずれ5分未満	☐

共有秘密鍵が漏れた場合

両サーバーの鍵を同時に新しい値へ変更し、PHP-FPMをreloadし、送信元・受信ログを確認します。古い鍵は直ちに無効化します。

内部から外部だけ

外部サーバーが内部SQLiteを参照するポートやVPN経路を作りません。内部から外部のreceive-availability.phpへHTTPS POSTするだけです。

完成後に残す記録

次の情報を組織の運用台帳へ登録すると、担当者が変わっても復旧できます。秘密値そのものは台帳の公開欄へ書きません。

記録項目	内容
稼働開始日	年 / 月 / 日、承認者、作業者
ソフト版	Gitコミット a4469ae、release 20260817-a4469ae
内部URL	HTTPS URL、アクセス可能なLAN/VPN
外部URL	https://外部FQDN/calendar
サーバー	ホスト名、IP、OS、管理部署、保守期限
データ	内部DBパス、外部JSONパス
秘密管理	共有秘密鍵と証明書秘密鍵の保管場所だけ
監視	内部Web、外部Web、health、timer、証明書期限
バックアップ	頻度、保存先、暗号化、保持期間、復元試験日
連絡先	アプリ担当、Linux担当、ネットワーク担当、証明書担当
緊急停止	内部Web停止、外部受信停止、鍵ローテーション手順

完了

A〜Hと第23章の全項目が合格し、運用台帳とバックアップが整った時点で構築完了です。

この手順書は、Ubuntu Server 24.04 LTS、Nginx、PHP-FPMを前提としています。別ディストリビューション、Apache、コンテナ、クラウド固有ロードバランサーを使う場合は、サーバー管理者向け手順へ切り替えてください。

END OF GUIDE